DESIGN AND IMPLEMENTATION OF NXN MULTIPLIER ARCHITECTURE BASED ON ROW-BYPASSING AND HYBRID ADDERS

By

M. ADHIYAMAAN	23BEC1439
JANANI VL	23BEC1406
BHARGAV SIMHA H	22BEC1273
MOHAMED SUHAIL A	23BEC1165

An Implementation report submitted to

Dr. S. UMADEVI

SCHOOL OF ELECTRONICS ENGINEERING

in partial fulfillment of the internship requirements at the

Centre for Nanoelectronics and VLSI Design

in

B.Tech. ELECTRONICS AND COMMUNICATION ENGINEERING



Vellore Institute of Technology (VIT) CHENNAI

Vandalur – Kelambakkam Road

Chennai – 600127

SEPTEMBER 2025

1. MODULE SUMMARY TABLES

1.1 Quadrant Multiplier with Hybrid Adders

Module Name	Sub-Modules Used	Purpose / Function
csa_bypass_fa	–	Carry-Save Adder (CSA) Full Adder Cell with bypass
mulNxN_csa_row_bypass	csa_bypass_fa	NxN Row-Bypassing Multiplier (PPG + CSA tree)
han_carlson_4bit	–	4-bit Han-Carlson Adder for fast prefix carry
weinberger_4bit	–	4-bit Weinberger Adder for intermediate blocks
ling_4bit	–	4-bit Ling Adder for delay reduction in final stage
hybrid_adder_nbit	han_carlson_4bit, weinberger_4bit, ling_4bit	N-bit Hybrid Adder combining three architectures
mul_quadrant_rowbypass_hybrid	mulNxN_csa_row_bypass, hybrid_adder_nbit	Final Multiplier: Quadrant approach + Hybrid adders

1.2 N-bit Hybrid Adder

Module Name	Sub-Modules Used	Purpose / Function
han_carlson_4bit	–	Fast carry using parallel-prefix logic
weinberger_4bit	–	Speed with balanced fan-out
ling_4bit	–	Reduced delay in final stage
hybrid_adder_nbit	han_carlson_4bit, weinberger_4bit, ling_4bit	Combines 4-bit blocks to form N-bit adder
nbit_hybrid_adder_tb	hybrid_adder_nbit	Testbench for multiple cases, checks correctness

1.3 Row-Bypassing Multiplier

Module Name	Sub-Modules Used	Purpose / Function
csa_bypass_fa	–	CSA Full Adder with bypass feature
csa_bypass_fa_right	csa_bypass_fa	Rightmost column carry fix in CSA row
fa_rca	–	Single-bit Full Adder for RCA stage
mulNxN_csa_row_bypass	csa_bypass_fa, csa_bypass_fa_right, fa_rca	Main Multiplier: partial products, row compression, sum
tb	mulNxN_csa_row_bypass	Testbench for verification (fixed + random inputs)

2. CODE IMPLEMENTATION

2.1 Quadrant Multiplier with Hybrid Adders Code

```
`timescale 1ns/1ps

// ============================================================
// Row-bypass Full-Adder Cell
// ============================================================
module csa_bypass_fa (
input  wire aibj,
input  wire s_in,
input  wire c_in,
input  wire bj,
output wire s_out,
output wire c_out
);
wire s_fa = aibj ^ s_in ^ c_in;
wire c_fa = (aibj & s_in) | (aibj & c_in) | (s_in & c_in);

assign s_out = bj ? s_fa : s_in;
assign c_out = bj ? c_fa : c_in;
endmodule
```

```
// ======================================================
// Structural Row-Bypassing NxN Multiplier
// ======================================================
module mulNxN_csa_row_bypass #(parameter N = 8)(
input  wire [N-1:0] a,
input  wire [N-1:0] b,
output wire [2*N-1:0] p
);
localparam M = 2 * N;
wire [M-1:0] acc_stage [0:N];
genvar gi, j, k, idx;

// Initialize accumulation stage to zero
generate
for (gi = 0; gi < M; gi = gi + 1) begin : INIT_ZERO
assign acc_stage[0][gi] = 1'b0;
end
endgenerate

// Row-wise accumulation of partial products
generate
for (j = 0; j < N; j = j + 1) begin : ROWS
wire [M-1:0] partial;

// Partial product generation for row j
for (idx = 0; idx < M; idx = idx + 1) begin : PART_GEN
if (idx >= j && idx < j + N)
assign partial[idx] = a[idx - j] & b[j];
else
assign partial[idx] = 1'b0;
end

wire [M:0] carry_row;
assign carry_row[0] = 1'b0;

// Column-wise full adders
for (k = 0; k < M; k = k + 1) begin : COLS
csa_bypass_fa FA_inst (
.aibj(partial[k]),
.s_in(acc_stage[j][k]),
.c_in(carry_row[k]),
```

```
.bj(b[j]),
.s_out(acc_stage[j+1][k]),
.c_out(carry_row[k+1])
);
end
end
endgenerate

// Final product output
assign p = acc_stage[N];
endmodule


// ============================================================
// 4-bit Adder Modules (Han-Carlson, Weinberger, Ling)
// ============================================================
module han_carlson_4bit(
input  [3:0] A, B,
input        Cin,
output [3:0] Sum,
output       Cout
);
wire [3:0] G, P;
wire [4:0] C;

assign G = A & B;
assign P = A ^ B;
assign C[0] = Cin;

assign C[1] = G[0] | (P[0] & C[0]);
assign C[2] = G[1] | (P[1] & C[1]);
assign C[3] = G[2] | (P[2] & C[2]);
assign C[4] = G[3] | (P[3] & C[3]);

assign Sum = P ^ C[3:0];
assign Cout = C[4];
endmodule


module weinberger_4bit(
input  [3:0] A, B,
```

```
input          Cin,
output [3:0] Sum,
output         Cout
);
wire [3:0] G, P, Gp, Pp;
wire [4:0] C;

assign G = A & B;
assign P = A ^ B;

assign Gp[0] = G[0]; assign Pp[0] = P[0];
assign Gp[1] = G[1] | (P[1] & Gp[0]); assign Pp[1] = P[1] & Pp[0];
assign Gp[2] = G[2] | (P[2] & Gp[1]); assign Pp[2] = P[2] & Pp[1];
assign Gp[3] = G[3] | (P[3] & Gp[2]); assign Pp[3] = P[3] & Pp[2];

assign C[0] = Cin;
assign C[1] = Gp[0] | (Pp[0] & Cin);
assign C[2] = Gp[1] | (Pp[1] & Cin);
assign C[3] = Gp[2] | (Pp[2] & Cin);
assign C[4] = Gp[3] | (Pp[3] & Cin);

assign Sum = P ^ C[3:0];
assign Cout = C[4];
endmodule


module ling_4bit(
input  [3:0] A, B,
input          Cin,
output [3:0] Sum,
output         Cout
);
wire [3:0] G, P, H;
wire [4:0] C;

assign G = A & B;
assign P = A ^ B;

assign H[0] = G[0];
assign H[1] = G[1] | (P[1] & G[0]);
assign H[2] = G[2] | (P[2] & H[1]);
```

```
assign H[3] = G[3] | (P[3] & H[2]);

assign C[0] = Cin;
assign C[1] = H[0] | (P[0] & C[0]);
assign C[2] = H[1] | (P[1] & P[0] & C[0]);
assign C[3] = H[2] | (P[2] & P[1] & P[0] & C[0]);
assign C[4] = H[3] | (P[3] & P[2] & P[1] & P[0] & C[0]);

assign Sum = P ^ C[3:0];
assign Cout = C[4];
endmodule


// ========================================================
// N-bit Hybrid Adder
// ========================================================
module hybrid_adder_nbit #(parameter N = 12)(
input  [N-1:0] A, B,
input          Cin,
output [N-1:0] Sum,
output         Cout
);
localparam NUM_BLOCKS = (N+3)/4;
localparam PAD      = NUM_BLOCKS * 4;

wire [PAD-1:0] A_ext = {{(PAD-N){1'b0}}, A};
wire [PAD-1:0] B_ext = {{(PAD-N){1'b0}}, B};
wire [PAD-1:0] S_ext;
wire [NUM_BLOCKS:0] carry;

assign carry[0] = Cin;
assign Cout = carry[NUM_BLOCKS];
assign Sum  = S_ext[N-1:0];

genvar i;
generate
for (i=0; i<NUM_BLOCKS; i=i+1) begin : BLK
wire [3:0] Ai = A_ext[i*4 +: 4];
wire [3:0] Bi = B_ext[i*4 +: 4];
wire [3:0] Si; wire Ci;
```

```
if (i%3==0) han_carlson_4bit u(Ai, Bi, carry[i], Si, Ci);
else if (i%3==1) weinberger_4bit u(Ai, Bi, carry[i], Si, Ci);
else ling_4bit u(Ai, Bi, carry[i], Si, Ci);

assign S_ext[i*4 +: 4] = Si;
assign carry[i+1] = Ci;
end
endgenerate
endmodule


// ======================================================
// Quadrant Multiplier with Hybrid Adders
// ======================================================
module mul_quadrant_rowbypass_hybrid #(parameter N = 16)(
input  wire [N-1:0] A,
input  wire [N-1:0] B,
output wire [2*N-1:0] P
);
localparam H = N/2;

// Partial Product Generation
wire [H-1:0] A_lo = A[H-1:0], A_hi = A[N-1:H];
wire [H-1:0] B_lo = B[H-1:0], B_hi = B[N-1:H];
wire [2*H-1:0] PR, PS, QR, QS;

mulNxN_csa_row_bypass #(.N(H)) uPR(A_lo, B_lo, PR);
mulNxN_csa_row_bypass #(.N(H)) uPS(A_lo, B_hi, PS);
mulNxN_csa_row_bypass #(.N(H)) uQR(A_hi, B_lo, QR);
mulNxN_csa_row_bypass #(.N(H)) uQS(A_hi, B_hi, QS);

// Shift and Sum
wire [2*N-1:0] term_PR      = PR;
wire [2*N-1:0] term_PS_shifted = PS << H;
wire [2*N-1:0] term_QR_shifted = QR << H;
wire [2*N-1:0] term_QS_shifted = QS << (2*H);

wire [2*N-1:0] sum_stage1, sum_stage2;

hybrid_adder_nbit #(.N(2*N)) adder_stage1 (
.A(term_PS_shifted),
```

```
.B(term_QR_shifted),
.Cin(1'b0),
.Sum(sum_stage1),
.Cout()
);

hybrid_adder_nbit #(.N(2*N)) adder_stage2 (
.A(sum_stage1),
.B(term_PR),
.Cin(1'b0),
.Sum(sum_stage2),
.Cout()
);

hybrid_adder_nbit #(.N(2*N)) adder_stage3 (
.A(sum_stage2),
.B(term_QS_shifted),
.Cin(1'b0),
.Sum(P),
.Cout()
);
endmodule
// ========================================================
// Testbench for N=16 Multiplier
// ========================================================
module tb_multiplier_N16;
localparam N = 16;
localparam M = 2 * N;

reg  [N-1:0] A, B;
wire [M-1:0] P, GOLD;

// Instantiate DUT
mul_quadrant_rowbypass_hybrid #(.N(N)) dut (.A(A), .B(B), .P(P));

assign GOLD = A * B;

initial begin
$display("--- Starting Multiplier Test for N=16 ---");

// Test vector
```

```
A = 16'hBEEF;
B = 16'hCAFE;
#10;

if (P !== GOLD)
$fatal(1, "TEST FAILED | A=%h, B=%h | DUT=%h, EXP=%h", A, B, P, GOLD);
else begin
$display("TEST PASSED!");
$display("A = %0d (%h)", A, A);
$display("B = %0d (%h)", B, B);
$display("P = %0d (%h)", P, P);
end
$finish;
end
endmodule
```

2.2 N-bit Hybrid Adder Code

```
//============================
// han_carlson_4bit.v - With realistic gate delays
//============================
`timescale 1ns/1ps
module han_carlson_4bit (
input  [3:0] A,
input  [3:0] B,
input        Cin,
output wire [3:0] Sum,
output wire  Cout
);
wire [3:0] G, P;
wire [4:0] C;

// Generate and Propagate - 1 gate delay each
assign #1 G = A & B;  // AND gate delay
```

```
assign #1 P = A ^ B;  // XOR gate delay

// Parallel prefix carry calculation - 2 gate delays per level
assign C[0] = Cin;
assign #2 C[1] = G[0] | (P[0] & C[0]);  // OR + AND gate
assign #2 C[2] = G[1] | (P[1] & C[1]);
assign #2 C[3] = G[2] | (P[2] & C[2]);
assign #2 C[4] = G[3] | (P[3] & C[3]);

// Sum calculation - 1 XOR gate delay
assign #1 Sum = P ^ C[3:0];
assign Cout = C[4];
endmodule

//============================
// weinberger_4bit.v - With realistic gate delays
//============================
`timescale 1ns/1ps
module weinberger_4bit (
input  [3:0] A,
input  [3:0] B,
input        Cin,
output wire [3:0] Sum,
output wire  Cout
);
wire [3:0] G, P;
wire [3:0] G_prefix, P_prefix;
wire [4:0] C;
```

```
// Generate and Propagate - 1 gate delay each
assign #1 G = A & B;
assign #1 P = A ^ B;

// Weinberger prefix computation - parallel levels
assign G_prefix[0] = G[0];
assign P_prefix[0] = P[0];
assign #2 G_prefix[1] = G[1] | (P[1] & G_prefix[0]);  // Level 1
assign #2 P_prefix[1] = P[1] & P_prefix[0];
assign #2 G_prefix[2] = G[2] | (P[2] & G_prefix[1]);  // Level 2
assign #2 P_prefix[2] = P[2] & P_prefix[1];
assign #2 G_prefix[3] = G[3] | (P[3] & G_prefix[2]);  // Level 3
assign #2 P_prefix[3] = P[3] & P_prefix[2];

// Carry generation - 2 gate delays
assign C[0] = Cin;
assign #2 C[1] = G_prefix[0] | (P_prefix[0] & Cin);
assign #2 C[2] = G_prefix[1] | (P_prefix[1] & Cin);
assign #2 C[3] = G_prefix[2] | (P_prefix[2] & Cin);
assign #2 C[4] = G_prefix[3] | (P_prefix[3] & Cin);

assign #1 Sum = P ^ C[3:0];
assign Cout = C[4];
endmodule

//============================
// ling_4bit.v - With realistic gate delays
//============================
`timescale 1ns/1ps
```

```
module ling_4bit (
input  [3:0] A,
input  [3:0] B,
input        Cin,
output wire [3:0] Sum,
output wire  Cout
);
wire [3:0] G, P;
wire [3:0] H;
wire [4:0] C;

// Generate and Propagate - 1 gate delay each
assign #1 G = A & B;
assign #1 P = A ^ B;

// Ling's H function computation - optimized for speed
assign H[0] = G[0];
assign #2 H[1] = G[1] | (P[1] & G[0]);      // 2 gate delays
assign #2 H[2] = G[2] | (P[2] & H[1]);      // 2 gate delays
assign #2 H[3] = G[3] | (P[3] & H[2]);      // 2 gate delays

// Carry generation - parallel structure for speed
assign C[0] = Cin;
assign #2 C[1] = H[0] | (P[0] & C[0]);
assign #2 C[2] = H[1] | (P[1] & P[0] & C[0]);    // Optimized path
assign #2 C[3] = H[2] | (P[2] & P[1] & P[0] & C[0]);
assign #2 C[4] = H[3] | (P[3] & P[2] & P[1] & P[0] & C[0]);

assign #1 Sum = P ^ C[3:0];
```

```
assign Cout = C[4];
endmodule

//==============================
// N-bit Hybrid Adder - Parameterized
//==============================
`timescale 1ns/1ps
module hybrid_adder_nbit #(
parameter N = 12  // Default 12-bit, can be changed
)(
input  [N-1:0] A,
input  [N-1:0] B,
input          Cin,
output [N-1:0] Sum,
output         Cout
);
// Calculate number of 4-bit blocks needed
localparam NUM_BLOCKS = (N + 3) / 4;  // Ceiling division
localparam PADDED_WIDTH = NUM_BLOCKS * 4;

// Padded inputs for complete 4-bit blocks
wire [PADDED_WIDTH-1:0] A_padded, B_padded;
wire [PADDED_WIDTH-1:0] Sum_padded;
wire [NUM_BLOCKS:0] carry_chain;

// Pad inputs with zeros for the upper bits
assign A_padded = {{(PADDED_WIDTH-N){1'b0}}, A};
assign B_padded = {{(PADDED_WIDTH-N){1'b0}}, B};
```

```
// Extract the actual N-bit result
assign Sum = Sum_padded[N-1:0];

// Initialize carry chain
assign carry_chain[0] = Cin;
assign Cout = carry_chain[NUM_BLOCKS];

// Generate 4-bit adder blocks
genvar i;
generate
for (i = 0; i < NUM_BLOCKS; i = i + 1) begin : adder_blocks
if (i % 3 == 0) begin : han_carlson_stage
// Use Han-Carlson for every 3rd block (0, 3, 6, ...)
han_carlson_4bit adder_inst (
.A(A_padded[i*4+3:i*4]),
.B(B_padded[i*4+3:i*4]),
.Cin(carry_chain[i]),
.Sum(Sum_padded[i*4+3:i*4]),
.Cout(carry_chain[i+1])
);
end else if (i % 3 == 1) begin : weinberger_stage
// Use Weinberger for every 3rd block + 1 (1, 4, 7, ...)
weinberger_4bit adder_inst (
.A(A_padded[i*4+3:i*4]),
.B(B_padded[i*4+3:i*4]),
.Cin(carry_chain[i]),
.Sum(Sum_padded[i*4+3:i*4]),
.Cout(carry_chain[i+1])
);
```

```
end else begin : ling_stage
// Use Ling for every 3rd block + 2 (2, 5, 8, ...)
ling_4bit adder_inst (
.A(A_padded[i*4+3:i*4]),
.B(B_padded[i*4+3:i*4]),
.Cin(carry_chain[i]),
.Sum(Sum_padded[i*4+3:i*4]),
.Cout(carry_chain[i+1])
);
end
end
endgenerate

endmodule

//==============================
// Fixed N-bit Hybrid Adder Testbench - Parameterized
//==============================
`timescale 1ns/1ps
module nbit_hybrid_adder_tb #(
parameter N = 8  // Test with 16-bit adder
);
reg  [N-1:0] A, B;
reg          Cin;
wire [N-1:0] Sum;
wire         Cout;

// Instantiate N-bit hybrid adder
hybrid_adder_nbit #(.N(N)) dut (
```

```
.A(A),
.B(B),
.Cin(Cin),
.Sum(Sum),
.Cout(Cout)
);

// Test variables
reg [N:0] expected_result;
integer test_count = 0;
integer pass_count = 0;

initial begin
$dumpfile("nbit_hybrid_adder.vcd");
$dumpvars(0, nbit_hybrid_adder_tb);

$display("=== N-bit Hybrid Adder Test (N=%0d) ===", N);
$display("");

// Initialize
A = 0; B = 0; Cin = 0;
#10;

// Test 1: Simple addition
$display("=== Test 1: Simple Addition ===");
test_count = test_count + 1;
A = 16'h1234;  // For 16-bit
B = 16'h5678;  // For 16-bit
Cin = 1'b0;
```

```
#20; // Wait for propagation

expected_result = A + B + Cin;

$display("Test %0d: A=%h, B=%h, Cin=%b", test_count, A, B, Cin);

$display("Expected: Sum=%h, Cout=%b", expected_result[N-1:0],
expected_result[N]);

$display("Actual:   Sum=%h, Cout=%b", Sum, Cout);

if (Sum == expected_result[N-1:0] && Cout == expected_result[N]) begin

$display("PASS ✓");

pass_count = pass_count + 1;

end else begin

$display("FAIL ✗");

end

$display("");


// Test 2: Addition with carry

$display("=== Test 2: Addition with Carry ===");

test_count = test_count + 1;

A = 16'hFFFF;  // Maximum value

B = 16'h0001;  // Add 1

Cin = 1'b0;

#20;

expected_result = A + B + Cin;

$display("Test %0d: A=%h, B=%h, Cin=%b", test_count, A, B, Cin);

$display("Expected: Sum=%h, Cout=%b", expected_result[N-1:0],
expected_result[N]);

$display("Actual:   Sum=%h, Cout=%b", Sum, Cout);

if (Sum == expected_result[N-1:0] && Cout == expected_result[N]) begin

$display("PASS ✓");

pass_count = pass_count + 1;

end else begin
```

```
$display("FAIL ✗");
end
$display("");

// Test 3: Addition with Cin = 1
$display("=== Test 3: Addition with Cin=1 ===");
test_count = test_count + 1;
A = 16'h7FFF;
B = 16'h8000;
Cin = 1'b1;
#20;
expected_result = A + B + Cin;
$display("Test %0d: A=%h, B=%h, Cin=%b", test_count, A, B, Cin);
$display("Expected: Sum=%h, Cout=%b", expected_result[N-1:0],
expected_result[N]);
$display("Actual:   Sum=%h, Cout=%b", Sum, Cout);
if (Sum == expected_result[N-1:0] && Cout == expected_result[N]) begin
$display("PASS ✓");
pass_count = pass_count + 1;
end else begin
$display("FAIL ✗");
end
$display("");

// Test 4: Zero addition
$display("=== Test 4: Zero Addition ===");
test_count = test_count + 1;
A = 16'h0000;
B = 16'h0000;
Cin = 1'b0;
```

```
#20;

expected_result = A + B + Cin;

$display("Test %0d: A=%h, B=%h, Cin=%b", test_count, A, B, Cin);

$display("Expected: Sum=%h, Cout=%b", expected_result[N-1:0],
expected_result[N]);

$display("Actual:   Sum=%h, Cout=%b", Sum, Cout);

if (Sum == expected_result[N-1:0] && Cout == expected_result[N]) begin

$display("PASS ✓");

pass_count = pass_count + 1;

end else begin

$display("FAIL ✗");

end

$display("");


// Test 5: Random test

$display("=== Test 5: Random Values ===");

test_count = test_count + 1;

A = 16'hABCD;

B = 16'h1234;

Cin = 1'b1;

#20;

expected_result = A + B + Cin;

$display("Test %0d: A=%h, B=%h, Cin=%b", test_count, A, B, Cin);

$display("Expected: Sum=%h, Cout=%b", expected_result[N-1:0],
expected_result[N]);

$display("Actual:   Sum=%h, Cout=%b", Sum, Cout);

if (Sum == expected_result[N-1:0] && Cout == expected_result[N]) begin

$display("PASS ✓");

pass_count = pass_count + 1;

end else begin
```

```
$display("FAIL ✗");
end
$display("");

// Final results
$display("=== Test Summary ===");
$display("Total Tests: %0d", test_count);
$display("Passed: %0d", pass_count);
$display("Failed: %0d", test_count - pass_count);
if (pass_count == test_count) begin
$display("ALL TESTS PASSED! ✓");
end else begin
$display("SOME TESTS FAILED! ✗");
end

$display("Simulation completed at time: %0t", $time);
#10;
$finish;
end

endmodule

// Top module for testing
module test_multiple_widths;
// Test 16-bit adder
nbit_hybrid_adder_tb #(.N(20)) tb_20bit();

endmodule
```

2.3 Row-Bypassing Multiplier Code

```
module csa_bypass_fa #(parameter USE_Z_TRISTATE=0)(
input  wire aibj,
input  wire s_in,
input  wire c_in,
input  wire bj,
output wire s_out,
output wire c_out
);
wire a_g = (USE_Z_TRISTATE) ? (bj ? aibj : 1'bz) : (bj ? aibj : 1'b0);
wire s_g = (USE_Z_TRISTATE) ? (bj ? s_in : 1'bz) : (bj ? s_in : 1'b0);
wire c_g = (USE_Z_TRISTATE) ? (bj ? c_in : 1'bz) : (bj ? c_in : 1'b0);
wire s_fa = a_g ^ s_g ^ c_g;
wire c_fa = (a_g & s_g) | (a_g & c_g) | (s_g & c_g);
assign s_out = bj ? s_fa : s_in;
assign c_out = bj ? c_fa : c_in;
endmodule

module csa_bypass_fa_right(
input  wire aibj,
input  wire s_in,
input  wire c_in,
input  wire bj,
output wire s_out,
output wire c_fix
);
wire s_tmp, c_tmp;
```

```
csa_bypass_fa U(.aibj(aibj), .s_in(s_in), .c_in(c_in), .bj(bj), .s_out(s_tmp),
.c_out(c_tmp));

assign s_out = s_tmp;

assign c_fix = bj ? c_tmp : (aibj & c_in);

endmodule


module fa_rca(input wire x, input wire y, input wire cin, output wire sum, output wire
cout);

assign sum  = x ^ y ^ cin;

assign cout = (x & y) | (x & cin) | (y & cin);

endmodule


module mulNxN_csa_row_bypass #(parameter N=8)(

input  wire [N-1:0] a,

input  wire [N-1:0] b,

output wire [2*N-1:0] p

);

wire [N-1:0] pp [0:N-1];

genvar gi, gj;

generate

for (gj=0; gj<N; gj=gj+1) begin: GEN_PP_ROW

for (gi=0; gi<N; gi=gi+1) begin: GEN_PP_COL

assign pp[gj][gi] = a[gi] & b[gj];

end

end

endgenerate


assign p[0] = (N>=1) ? pp[0][0] : 1'b0;


wire s10 = (N>1) ? (pp[0][1] ^ pp[1][0]) : 1'b0;
```

```
wire c10 = (N>1) ? (pp[0][1] & pp[1][0]) : 1'b0;

assign p[1] = (N>1) ? s10 : 1'b0;


wire s [0:N-1][0:N-1];

wire c [0:N-1][0:N-1];


if (N>2) begin: ROW1

csa_bypass_fa U11 (.aibj(pp[1][1]), .s_in(pp[0][2]), .c_in(c10), .bj(b[1]),
.s_out(s[1][1]), .c_out(c[1][1]));

for (gi=2; gi<N-1; gi=gi+1) begin: R1_MID

csa_bypass_fa U1i (.aibj(pp[1][gi]), .s_in(pp[0][gi+1]), .c_in(c[1][gi-1]), .bj(b[1]),
.s_out(s[1][gi]), .c_out(c[1][gi]));

end

csa_bypass_fa_right U1R (.aibj(pp[1][N-1]), .s_in(1'b0), .c_in((N>2)?c[1][N-2]:c10),
.bj(b[1]), .s_out(s[1][N-1]), .c_fix(c[1][N-1]));

end


generate

if (N>3) begin: ROWS_MID

for (gj=2; gj<N-1; gj=gj+1) begin: ROWJ

csa_bypass_fa Uj1 (.aibj(pp[gj][1]), .s_in(s[gj-1][1]), .c_in(1'b0), .bj(b[gj]),
.s_out(s[gj][1]), .c_out(c[gj][1]));

for (gi=2; gi<N-1; gi=gi+1) begin: UJ_MID

csa_bypass_fa Uji (.aibj(pp[gj][gi]), .s_in(s[gj-1][gi]), .c_in(c[gj][gi-1]), .bj(b[gj]),
.s_out(s[gj][gi]), .c_out(c[gj][gi]));

end

csa_bypass_fa_right UjR (.aibj(pp[gj][N-1]), .s_in(s[gj-1][N-1]), .c_in(c[gj][N-2]),
.bj(b[gj]), .s_out(s[gj][N-1]), .c_fix(c[gj][N-1]));

end

end

endgenerate
```

```
generate

if (N>=3) begin: ROW_LAST

csa_bypass_fa UL1 (.aibj(pp[N-1][1]), .s_in((N==3)?s[1][1]:s[N-2][1]), .c_in(1'b0),
.bj(b[N-1]), .s_out(s[N-1][1]), .c_out(c[N-1][1]));

for (gi=2; gi<N-1; gi=gi+1) begin: UL_MID

csa_bypass_fa ULi (.aibj(pp[N-1][gi]), .s_in(s[N-2][gi]), .c_in(c[N-1][gi-1]), .bj(b[N-1]),
.s_out(s[N-1][gi]), .c_out(c[N-1][gi]));

end

csa_bypass_fa_right ULR (.aibj(pp[N-1][N-1]), .s_in((N==3)?s[1][N-1]:s[N-2][N-1]),
.c_in(c[N-1][N-2]), .bj(b[N-1]), .s_out(s[N-1][N-1]), .c_fix(c[N-1][N-1]));

end

endgenerate


wire [2*N-1:2] X, Y;


assign X[2] = (N>=3) ? s[N-1][1] : 1'b0;

assign Y[2] = 1'b0;


genvar k;

generate

for (k=3; k<=2*N-2; k=k+1) begin: BUILD_RCA_TERMS

localparam integer COL = ((k-1) <= (N-1)) ? (k-1) : (N-1);

localparam integer ROW = ((k-1) <= (N-1)) ? (N-1) : (2*N-2 - (k-1));

wire sx = ((ROW>=1)&&(ROW<=N-1)&&(COL>=1)&&(COL<=N-1)) ? s[ROW][COL] :
1'b0;

wire cy = (((ROW-1)>=1)&&((ROW-1)<=N-1)&&(COL>=1)&&(COL<=N-1)) ? c[ROW-
1][COL] : 1'b0;

assign X[k] = sx;

assign Y[k] = cy;

end

endgenerate
```

```
wire [2*N-2:2] cF;
fa_rca F2 (.x(X[2]), .y(Y[2]), .cin(1'b0), .sum(p[2]), .cout(cF[2]));
generate
for (k=3; k<=2*N-2; k=k+1) begin: MERGE
fa_rca Fk (.x(X[k]), .y(Y[k]), .cin(cF[k-1]), .sum(p[k]), .cout(cF[k]));
end
endgenerate
assign p[2*N-1] = cF[2*N-2];
endmodule

module tb;
localparam integer N = 16;
reg  [N-1:0] A, B;
wire [2*N-1:0] P;

mulNxN_csa_row_bypass #(.N(N)) dut (.a(A), .b(B), .p(P));

initial begin
A = 16'h0000; B = 16'h0000; #5;
A = 16'hF9E3; B = 16'h090B; #5;
A = 16'h1234; B = 16'hABCD; #5;
repeat (10) begin
A = $urandom; B = $urandom; #5;
end
#20 $finish;
end
endmodule
```